

PROJET SmartRoom

Hugo SCHENK - Antoine NAVETTE - Livia KOLETZKI

PRÉSENTATION DU PROJET

Introduction, cadre du projet, analyse du besoin métier, modélisation et choix architecturaux.

CONTEXTE & CAHIER DES CHARGES

Le Besoin Client

- Nécessité d'améliorer la rapidité de réservation des salles.
- Besoin urgent de traçabilité des réservations.
- Vision d'ensemble manquante (taux d'utilisation, localisation, équipements disponibles).

La Solution SmartRoom

- Centralisation de la gestion via une interface unique.
- Consultation des disponibilités en temps réel.
- Réservation, suivi et signalement d'incidents fluides.
- Accès administrateur pour l'édition de statistiques.

PRÉSENTATION DU PROJET

ANALYSE FONCTIONNELLE: DROITS D'ACCÈS

Rôle	Accès et actions autorisées
Visiteur	Lecture seule : Peut consulter la liste des salles, leurs équipements et disponibilités. Ne peut pas réserver.
User (Connecté)	Rechercher, réserver, modifier ou annuler ses propres réservations. Peut signaler un problème et gérer ses favoris.
Administrateur	Tous les droits : Création/suppression de salles, gestion des ressources, traitement des signalements, mise en indisponibilité, statistiques.

PRÉSENTATION DU PROJET

ANALYSE & CONCEPTION (MODÉLISATION)

Du Cahier des charges à la Base de données

Pour répondre précisément aux exigences, nous avons réalisé un travail complet de conception :

- Diagrammes de Cas d'Utilisation pour définir le périmètre des acteurs.
- Diagrammes de Séquence pour les flux complexes (ex: gestion des signalements).
- Élaboration du MCD (Modèle Conceptuel de Données) transformé ensuite en MPD pour notre base PostgreSQL.

ARCHITECTURE LOGICIELLE & CHOIX TECHNIQUES



Architecture en Couches

Séparation nette entre logique métier, données et interface (MVC amélioré).

Garantit l'indépendance et la testabilité.



Facteurs de Choix

Priorité à la **Performance (4/5)**, **Évolutivité & maintenabilité (3/5)** et **Simplicité (3/5)**.

L'interopérabilité (SSO) et le budget ont orienté vers une solution sur-mesure.



Gestion des Risques

Évitement des architectures surdimensionnées (Microservices) pour privilégier un monolithe modulaire adapté à notre équipe restreinte.

GESTION DE PROJET

Equipe, répartition du travail, organisation...

PRÉSENTATION ÉQUIPE



Livia



Hugueau



Antoine



Frédéric (RIP)

INGÉNIERIE BACK-END

Écosystème, conception de l'API RESTful et conformité
Twelve-Factor App.

INGÉNIERIE BACK-END

ECOSYSTÈME BACK-END

Robustesse et Typage Fort

Le cœur de l'application SmartRoom repose sur un backend moderne conçu pour la réactivité :

- **Langage** : TypeScript pour la sécurité de typage.
- **Framework** : NodeJS avec Fastify pour un traitement asynchrone performant.
- **Bases de données** : PostgreSQL pour le stockage relationnel.



PostgreSQL

CONCEPTION DE L'API RESTFUL

Standards HTTP Stricts

L'API expose des routes JSON standardisées (CRUD).

Utilisation rigoureuse des codes statuts :

- **Succès** : 200 OK, 201 Created, 204 No Content.
- **Erreurs** : 400 (Bad Request), 401/403 (Auth), 404 (Not Found), 409 (Conflict - ex: salle déjà réservée).

Documentation & Swagger

API entièrement documentée.

Utilisation d'**OpenAPI (Swagger)** accessible via /docs#/ , en code first.

→ contrat d'interface clair, facilitant considérablement le travail d'intégration avec le Front-End.

PRÉSENTATION STRUCTURE

11 services :

- auth
- user
- building
- floor
- room
- resource
- reservation
- favorite
- incident
- room-resource-assignment
- room-unavailability

Request lifecycle :

- Routes
- Schémas
- Services
- Repositories
- Entities

INGÉNIERIE BACK-END

AUTHENTIFICATION

- usage des sessions, pas des JWT
- stockées en BDD côté serveur
- token stockés dans un cookie côté client
- excellent compromis entre sécurité et simplicité

INGÉNIERIE FRONT-END

Architecture de l'application cliente, asynchronisme et
UX/UI.

ARCHITECTURE FRONT-END & DESIGN

Plannification

Lecture et analyse des diagrammes permettant une planification des futurs composants répondant à des cas d'utilisations spécifiques.



Maquettage UX/UI

Planification visuelle des vues, des fonctionnalités, des différents composants, hiérarchie des éléments, dispositions et rendu responsive.



Intégration front-end / consommation de l'API

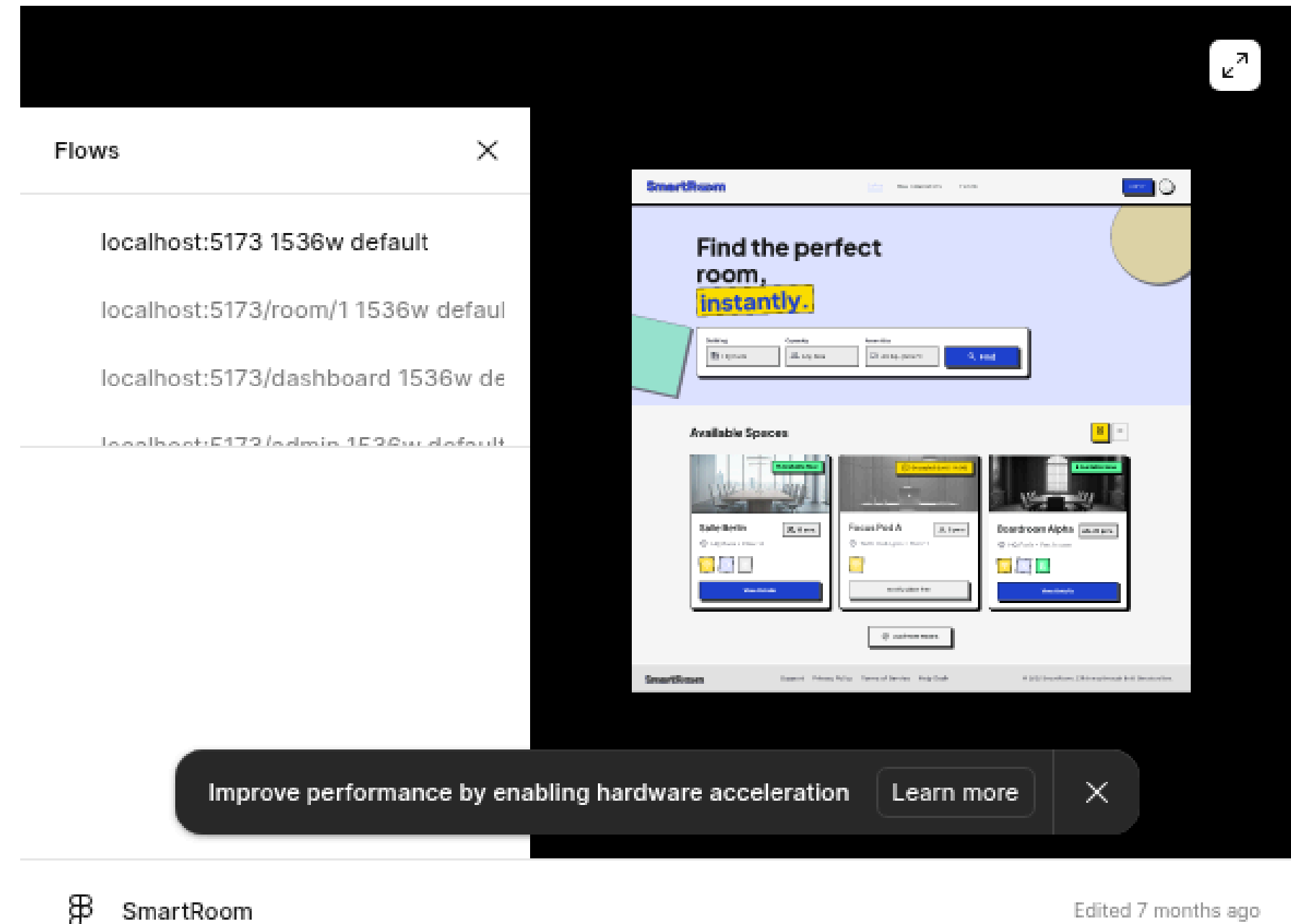
Transformation de la maquette en application fonctionnelle en consommant l'API, développement des fonctionnalités front-end et création de tests unitaires.



INGÉNIERIE FRONT-END

MAQUETTAGE UX/UI

- Création des composants
- Mise en place des différentes vues
- Choix du design et des couleurs
- Vérification de l'accessibilité
- Planification des dispositions en responsive design



NEOBRUTALISME EN FORCE !

INGÉNIERIE FRONT-END

INTÉGRATION FRONT-END /

CONSOMMATION DE L'API

- Intégration pas à pas des composants et pages
- Mise en place du style / responsive design avec Tailwind CSS
- Mise en place des routes
- Typage des données API
- Récupération des données et affichage
- Mise en place de tests unitaires

<https://smart-room.antoine-navette.fr>



CONSOMMATION API & ASYNCHRONISME

Gestion de l'Asynchronisme

L'application cliente effectue des requêtes asynchrones pour récupérer les disponibilités en temps réel, sans jamais bloquer l'interface utilisateur (UI).

Utilisation de hooks personnalisés et de mécanismes de "loading states" pour une expérience fluide pendant la récupération des données.

Traitement des Erreurs

La robustesse du Front-End repose sur la bonne gestion des réponses de l'API RESTful.

Interception des erreurs HTTP (4xx, 5xx) pour afficher des messages "Toast" clairs à l'utilisateur (ex: *"Cette salle vient d'être réservée"* ou *"Droits insuffisants"*).

ARCHITECTURE FRONT-END & DESIGN



Technologies

Client développé en **React** (SPA) pour une interface dynamique sans rechargement. Typage fort avec **TypeScript** garantissant l'alignement avec les modèles du Back-End.



Charte Graphique

Utilisation de **Tailwind CSS** pour implémenter notre propre Design System. Approche "Mobile-First" assurant une consultation fluide sur smartphone ou desktop.



Composants Clés

Développement de vues complexes : Calendrier interactif, moteur de recherche à multi-filtres (capacité, équipement), et tableau de bord administrateur clair.

DEVOPS & INFRASTRUCTURE

Conteneurisation, CI/CD, Tests et démarche qualité.

DEVOPS & CONTENEURISATION

Standardisation des Environnements

Pour assurer la fiabilité du déploiement (livrable final) et respecter la parité Dev/Prod, l'infrastructure est entièrement conteneurisée :

- **Docker** : Création d'images isolées pour le Front, le Back et les services tiers.
- **Docker Compose** : Orchestration locale permettant de lancer toute la stack en une seule commande (Base de données, API, Interface).

CI/CD, TESTS ET GESTION DE PROJET



GIT

Travail collaboratif structuré via Github. Utilisation stricte des branches (feature branches) et des Pull Requests pour la revue de code avant l'intégration.



Stratégie CI/CD

Mise en place de pipelines d'Intégration Continue (CI) automatisant le build du code et l'exécution des tests à chaque push pour sécuriser nos livraisons (CD).



Qualité & Tests

Développement de tests couvrant la logique métier (Back-End) et les composants critiques (Front-End) assurant le respect du cahier des charges et des normes.

CONFORMITÉ

"TWELVE-FACTOR APP"

1. **Base de code** : usage de Git et GitHub
2. **Dépendances** : package.json côté front comme back
3. **Configuration** : usage de .env pour configurer ce qui doit l'être
4. **Services externes** : SMTP et BDD configurés via le .env
5. **Build, release, run** : géré via pipeline
6. **Processus** : rien n'est stocké directement sur le serveur

CONFORMITÉ

"TWELVE-FACTOR APP"

- 7. **Liaison de port** : application autonome, ports configurables
- 8. **Concurrence** : non gérée
- 9. **Jetable** : présence d'un graceful shutdown et démarrage rapide
- 10. **Parité dev/prod** : staging et prod similaires hors config, dev quasiment
- 11. **Logs** : présence de logs envoyées vers stdout
- 12. **Processus d'administration** : scripts de migrations lancés depuis un conteneur en dev et prod

DÉMONSTRATION TECHNIQUE

Présentation interactive de l'application SmartRoom V1
en conditions réelles.

PÉRIMÈTRE FONCTIONNEL

- ✓ **Livrables V1** : Authentification, CRUD Salles/Ressources, Recherche avancée avec filtres, Réservation simple, Gestion des favoris, Gestion des incidents, Droits d'accès basiques et Documentation Swagger.
- **En cours d'intégration** : Vue calendrier interactive avancée, système complet de notifications par mail interne, traitement affiné des signalements d'incidents et analyses statistiques complexes.
- ★ **Cibles Futures** : Intégration SSO externe complète de l'entreprise, synchro avec calendriers externes (Outlook/Google) et gestion du profil avancée.

CONCLUSION

Évolutions futures, Roadmap et Retour d'expérience de l'équipe.

APPROCHE GBB

GOOD	BETTER	BEST
• Création de réservations en temps réel • Authentification & rôles • Gestion avancée des bâtiments/étages/salles/ressources • Recherche avec filtres • Gestion des incidents	• Statistiques & rapports d'occupation, • Plan interactif, • Notifications, • Guide utilisateurs	• Récurrence de réservations, • Favoris, • Intégration calendrier (.Cal)

ÉVOLUTIVITÉ & FUTURES VERSIONS ENVISAGEABLES

Le Plan Interactif

Conformément aux spécifications, la prochaine étape majeure est l'intégration d'un plan interactif.

Permettra aux utilisateurs de sélectionner des salles directement sur une carte 2D/3D des bâtiments, offrant une vision spatiale immédiate des disponibilités.

Statistiques

Il s'agit d'une demande importante du client.

Indicateurs clés à produire :

- Le taux d'occupation global et par salle.
- Le volume de réservations par période.
- L'identification des créneaux de "pics d'utilisation".

RETOUR D'EXPÉRIENCE

1. Phase de conception

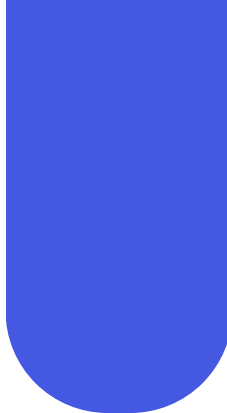
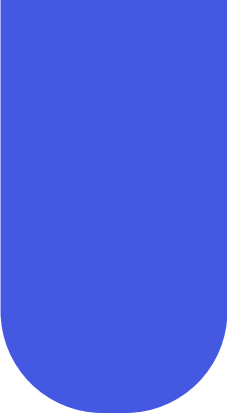
- Résister à l'envie de "coder immédiatement"
- Traduire un besoin client brut en MCD, MPD et diagrammes UML

2. Phase d'arbitrage

- La confrontation entre un Cahier des charges ambitieux et le chronomètre
- L'apprentissage de la priorisation : approche GBB

3. Phase de réalisation

- Synchroniser 3 profils aux affinités radicalement différentes (DevOps / Back / Front)
- La nécessité de "contrats d'interface" (Schémas Zod / Swagger)



MERCI !